

Simulação de Rede em Chip SPIN com SystemC

Flavio Oliveira Silva Junior, Leonardo Machado Moreira, Claudivan Costa de Lima Junior

Departamento de Informática e Matemática Aplicada, UFRN, Natal, Brasil

Resumo: Este trabalho apresenta o desenvolvimento de um simulador de Rede em Chip (Network-on-Chip, NoC) inspirado na arquitetura SPIN (Scalable Programmable Integrated Network), implementado em C++17 com SystemC. A rede é composta por 8 roteadores RSPIN organizados em uma topologia de árvore gorda quaternária simplificada de 2 níveis e 16 terminais. O simulador representa os pacotes em nível abstrato, usando terminais de origem e destino, buffers por porta, buffers centrais de overflow e controle de fluxo simplificado por capacidade de buffer. Os resultados do cenário principal demonstraram entrega de 100% dos pacotes, latência de 3 ciclos por pacote, 2 saltos de roteamento por mensagem e parada antecipada da simulação quando a rede ficou vazia.

Palavras-chave: Network-on-Chip; SPIN; árvore gorda quaternária; SystemC; simulação ciclo a ciclo; topologia K4,4.

I. INTRODUÇÃO

Com o aumento da complexidade dos sistemas integrados, os barramentos tradicionais passaram a apresentar limitações de escalabilidade, compartilhamento do meio e desempenho. As Redes em Chip (Network-on-Chip - NoC) surgiram como solução, interligando processadores, memórias e periféricos por meio de roteadores e enlaces dedicados, permitindo comunicações simultâneas e maior eficiência [1].

A arquitetura SPIN (Scalable Programmable Integrated Network) destaca-se pela organização em árvore gorda (fat-tree) e pelo uso de comunicação baseada em pacotes, com baixa latência, boa escalabilidade e mecanismos eficientes de roteamento e controle de fluxo [2]. Desenvolvida no LIP6/ASIM (Paris), a SPIN define roteadores RSPIN com portas inferiores (DN, conectadas a terminais ou roteadores filhos) e superiores (UP, conectadas ao roteador pai), formando uma hierarquia que elimina gargalos de barramento.

Este trabalho apresenta o desenvolvimento de um simulador da rede SPIN em SystemC [4], composto por 8 roteadores em topologia de árvore gorda quaternária simplificada e 16 terminais. O simulador lê a topologia e os eventos de tráfego a partir de arquivos texto, injeta pacotes na rede e simula sua propagação ciclo a ciclo, produzindo logs com rotas, latências, pacotes pendentes e estatísticas de desempenho.

II. REFERENCIAL TEÓRICO

A. Redes em Chip (NoC)

As NoCs substituem barramentos compartilhados por redes de interconexão com roteadores e enlaces dedicados, permitindo múltiplas comunicações simultâneas. Isso aumenta a largura de banda efetiva e reduz a latência em chips multiprocessados modernos [1].

B. Arquitetura SPIN e Árvore Gorda

A arquitetura SPIN organiza seus roteadores em uma árvore gorda quaternária. Em uma árvore gorda, os enlaces próximos à raiz têm maior capacidade agregada do que em uma árvore convencional, evitando gargalo no topo da hierarquia [2]. Na configuração simplificada de 8 roteadores usada neste trabalho, a rede é estruturada em dois níveis: quatro roteadores folha (R0-R3), cada um conectado a quatro terminais, e quatro roteadores de topo (R4-R7), cada um interligado a todos os roteadores folha. A conectividade é total entre os dois níveis:

cada folha R_i conecta suas portas U0-U3 respectivamente a R4, R5, R6 e R7; cada topo $R(4+j)$ conecta sua porta D_j ao roteador folha R_j .

C. Chaveamento Wormhole

No chaveamento Wormhole, o pacote é dividido em flits. O header flit reserva o caminho desde a origem até o destino, e os flits subsequentes seguem pela mesma rota sem necessidade de armazenamento completo em cada roteador. Isso reduz a latência em comparação ao modo store-and-forward, no qual o pacote inteiro precisa ser armazenado antes de ser retransmitido [3]. No modelo implementado neste trabalho, esse mecanismo é usado como referência conceitual, mas os pacotes são tratados como unidades atômicas, sem modelagem de flits individuais.

D. Roteamento Adaptativo e Determinístico

A SPIN emprega roteamento híbrido. Na subida (folha para topo), o roteamento pode escolher entre portas superiores disponíveis, enquanto na descida (topo para folha) o destino determina o roteador folha a ser alcançado [2]. Neste simulador, essa ideia é representada por uma política simplificada: a subida escolhe um roteador de topo disponível segundo a política interna do roteador, e a descida seleciona deterministicamente a folha associada ao terminal de destino.

E. Controle de Fluxo por Créditos

O controle de fluxo por créditos previne overflow de buffers sem retransmissão. Cada canal mantém um contador que reflete o espaço livre no buffer de entrada do vizinho. O emissor só transmite quando há espaço disponível no receptor [3]. Neste trabalho, o controle de fluxo é abstraído por capacidade de buffers: pacotes são aceitos quando há espaço e, em caso de saturação da entrada, podem ser mantidos nos buffers centrais QUP ou QDN para nova tentativa em ciclos seguintes.

III. MODELO IMPLEMENTADO

A. Estrutura Modular

O simulador é organizado em cinco módulos principais:

- **Packet:** struct que armazena id, terminal de origem, terminal de destino, roteadores correspondentes, ciclo de criação, ciclo de entrega e vetor route_history com o caminho percorrido.
- **SpinRouter:** modelo simplificado do roteador RSPIN com portas DN (D0-D3) e UP (U0-U3) representadas

por buffers de pacotes. Cada entrada possui capacidade para 4 pacotes, e os buffers centrais QUP e QDN possuem capacidade para 18 pacotes, usados como overflow quando a entrada principal está cheia.

- **SpinNetwork:** carrega a topologia, conecta os roteadores, coordena a injeção de pacotes e executa os ciclos de simulação em `step()`. Também reúne estatísticas de pacotes entregues, descartados e pendentes.
- **ConfigLoader:** conjunto de funções que lê os arquivos de topologia e tráfego. O arquivo de topologia define roteadores folha, roteadores de topo e terminais; o arquivo de tráfego usa a seção `[traffic]`, com campos: `ciclo`, `terminal_origem` e `terminal_destino`.
- **SpinSimulation:** módulo SystemC principal com `SC_THREAD` que executa o loop de simulação ciclo a ciclo, invocando `injectPacket()` e `step()`, e escreve o log de saída.

B. Topologia Implementada

A topologia de 8 roteadores e 16 terminais é carregada a partir do arquivo `input/topology_8.txt`. A Fig. 1 ilustra a hierarquia:

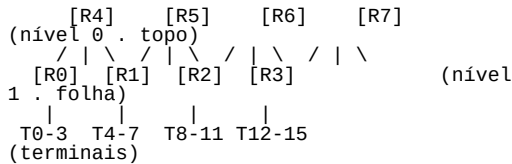


Fig. 1: Topologia em árvore gorda quaternária de 2 níveis

Cada roteador folha R_i ($i=0..3$) conecta suas portas $U0-U3$ respectivamente a $R4-R7$. Cada roteador de topo $R(4+j)$ conecta sua porta D_j à folha R_j . Terminais são numerados $T0-T15$: a folha R_i serve $T(4i)$ até $T(4i+3)$. Essa estrutura garante que qualquer par de terminais em folhas distintas é alcançável em 2 saltos de roteamento entre roteadores, além das etapas de injeção e entrega nos terminais.

C. Ciclo de Simulação

A cada ciclo, `SpinNetwork::step()` executa a sequência abaixo, mantendo separação entre o estado do ciclo atual e os pacotes visíveis no ciclo seguinte:

1. **Injeção:** os eventos do ciclo atual são lidos do arquivo de tráfego e os pacotes são inseridos no roteador folha associado ao terminal de origem.
2. **Processamento:** cada roteador examina suas entradas e seus buffers centrais, tenta encaminhar pacotes e registra requisições de entrega ou movimentação.
3. **Atualização:** pacotes encaminhados são inseridos nos buffers do próximo ciclo dos roteadores vizinhos, ou marcados como entregues quando alcançam o terminal de destino.
4. **Finalização do ciclo:** `commitNextCycle()` promove os buffers `next_` para `current_`. Se todos os eventos foram processados e a rede está vazia, a simulação encerra antecipadamente.

IV. RESULTADOS

A. Cenário de Simulação

O cenário de teste (`traffic_1.txt`) define 5 pacotes entre folhas distintas, injetados um por ciclo a partir do ciclo 0. A Tab. I apresenta as rotas e latências observadas no log de simulação.

Tab. I: Rotas e latências dos pacotes simulados

Pac.	Origem	Destino	Criado	Rota	Latência
P0	T0 → R0	T15 → R3	C0	R0 → R4 → R3	3 ciclos
P1	T5 → R1	T10 → R2	C1	R1 → R4 → R2	3 ciclos
P2	T14 → R3	T1 → R0	C2	R3 → R4 → R0	3 ciclos
P3	T3 → R0	T12 → R3	C3	R0 → R5 → R3	3 ciclos
P4	T8 → R2	T7 → R1	C4	R2 → R4 → R1	3 ciclos

B. Diagrama Temporal da Simulação

A Tab. II mostra o estado de cada pacote em cada ciclo de simulação, extraído diretamente do log. A tabela registra a injeção, a subida até o roteador de topo, a descida para a folha de destino e a entrega ao terminal final.

Tab. II: Diagrama temporal da simulação

	C0	C1	C2	C3	C4	C5	C6	C7
P0	Inj	↑R4	↓R3	√T15	.	.	.	.
P1	.	Inj	↑R4	↓R2	√T10	.	.	.
P2	.	.	Inj	↑R4	↓R0	√T1	.	.
P3	.	.	.	Inj	↑R5	↓R3	√T12	.
P4	.	.	.	.	Inj	↑R4	↓R1	√T7

O diagrama confirma o comportamento em 3 ciclos para o cenário principal: injeção em C, subida em C+1, descida em C+2 e entrega em C+3. P3 utiliza R5 em vez de R4 devido à política simplificada de seleção de saída superior do simulador. Esse comportamento distribui parte do tráfego entre roteadores de topo, mas não representa uma arbitragem SPIN completa.

C. Estatísticas Globais

A Tab. III resume as estatísticas extraídas do relatório final do simulador.

Tab. III: Estatísticas globais da simulação

Métrica	Valor
Pacotes injetados	5
Pacotes descartados	0
Pacotes entregues	5 (100 %)
Pacotes pendentes	0
Latência mínima	3 ciclos
Latência máxima	3 ciclos
Latência média	3,0 ciclos

Média de saltos (hops)	2,0
Throughput	0,625 pac./ciclo
Ciclos simulados	7

[4] IEEE, IEEE Std 1666-2023: Standard for Standard SystemC Language Reference Manual. New York: IEEE, 2023.

D. Trecho do Log de Simulação

O trecho abaixo, referente ao ciclo 3, ilustra a entrega do pacote P0 e o processamento de mais de uma operação dentro do mesmo ciclo:

```
===== CICLO 3 =====
[R3] DN2 sobe P2 -> R4 (U0)
[R3] UP0 entrega P0 ao terminal T15
[R4] DN1 desce P1 -> R2 (D2)
>>> P0 ENTREGUE ao T15 em R3
      latencia=3 ciclos | hops=2
```

R3 processa a subida de P2 (via DN2) e a entrega de P0 (via UP0) no mesmo ciclo de simulação. A latência de 3 ciclos no cenário principal corresponde a 2 saltos de roteamento entre roteadores e uma etapa adicional de entrega do pacote ao terminal de destino.

V. CONCLUSÃO

Este trabalho desenvolveu e validou um simulador de Rede em Chip inspirado na arquitetura SPIN, implementado em C++17 com SystemC. A topologia de árvore gorda quaternária simplificada com 8 roteadores RSPIN e 16 terminais foi representada com conexões ponto-a-ponto entre roteadores folha e roteadores de topo. O modelo considera pacotes em nível abstrato, buffers por porta, buffers centrais QUP e QDN para overflow e controle de fluxo simplificado por capacidade de buffer.

Os resultados confirmaram a operação correta do modelo no cenário principal: todos os 5 pacotes foram entregues sem descartes ou pendentes, com latência de 3 ciclos e 2 saltos de roteamento por mensagem. No cenário de estresse, o simulador também entregou todos os 15 pacotes, sem descartes e sem pendentes, com latência média de 4,4 ciclos, demonstrando que os buffers centrais são reprocessados corretamente em situações de maior contenção.

Como trabalhos futuros, propõe-se a implementação de flits reais para representar chaveamento Wormhole com maior fidelidade, controle de fluxo por créditos em nível de canal, arbitragem Round-Robin completa por porta de saída, geração de formas de onda VCD e extensão para topologias com mais níveis e maior número de roteadores.

REFERÊNCIAS

- [1] W. J. Dally e B. P. Towles, *Principles and Practices of Interconnection Networks*. San Francisco: Morgan Kaufmann, 2004.
- [2] F. Peyrard, J. Farré e M. Diaz, “SPIN: a Scalable Programmable Integrated Network,” *Proc. Euromicro Conf. Digital System Design*, Warsaw, 2001, pp. 102-109.
- [3] L. Benini e G. De Micheli, “Networks on Chips: A New SoC Paradigm,” *IEEE Computer*, vol. 35, n. 1, pp. 70-78, jan. 2002.